

Laxmi Charitable Trust's
Sheth L.U.J College of Arts & Sir M.V. College of Science and Commerce
Department of Information Technology (B.Sc.I.T Semester V)
APPLIED BUSINESS ANALYTICS
Practical –X

Roll No.:T053	Name:Nidhi Yadav
Class:TYIT	Batch:01
Date of Assignment:03-10-2026	Date/Time of Submission: 03-10-2026

Part - A :-- Regression Analysis

1. Load the sales_data.csv dataset.

```
Code: import pandas as pd
import matplotlib.pyplot as plt
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LinearRegression
from sklearn.metrics import r2_score, mean_squared_error
df = pd.read_csv("sales_data.csv")
```

2. Display the first five records.

```
print("First Five Records:")
```

```
print(df.head())
```

```
First Five Records:
  Product_ID  Sale_Date Sales_Rep Region  Sales_Amount  Quantity_Sold \
0      1052  2023-02-03      Bob  North      5053.97           18
1      1093  2023-04-21      Bob  West      4384.02           17
2      1015  2023-09-21     David  South      4631.23           30
3      1072  2023-08-24      Bob  South      2167.94           39
4      1061  2023-03-24     Charlie  East      3750.20           13

  Product_Category  Unit_Cost  Unit_Price Customer_Type  Discount \
0      Furniture      152.75      267.22      Returning      0.09
1      Furniture     3816.39     4209.44      Returning      0.11
2           Food      261.56      371.40      Returning      0.20
3      Clothing     4330.03     4467.75           New      0.02
4      Electronics     637.37     692.71           New      0.08

  Payment_Method Sales_Channel Region_and_Sales_Rep
0          Cash          Online          North-Bob
1          Cash          Retail          West-Bob
2  Bank Transfer          Retail        South-David
3   Credit Card          Retail        South-Bob
4   Credit Card          Online        East-Charlie
```

3. Identify the independent and dependent variables.

```
print("\nAvailable Columns:")
```

```
print(df.columns.tolist())
```

```
Available Columns:
['Product_ID', 'Sale_Date', 'Sales_Rep', 'Region', 'Sales_Amount', 'Quantity_Sold', 'Product_Category', 'Unit_Cost', 'Unit_Price', 'Customer_Type', 'Discount', 'Pay
```

4. Select advertising as the independent variable and sales as the dependent variable.

```
X = df[["Quantity_Sold"]]
```

```
y = df["Sales_Amount"]
```

```
Independent Variable: Quantity_Sold
Dependent Variable: Sales_Amount
```

```
print("\nIndependent Variable: Quantity_Sold")
```

```
print("Dependent Variable: Sales_Amount")
```

5. Check the data for missing values.

```
print("\nMissing Values:")
```

```
print(df[["Quantity_Sold", "Sales_Amount"]].isnull().sum())
```

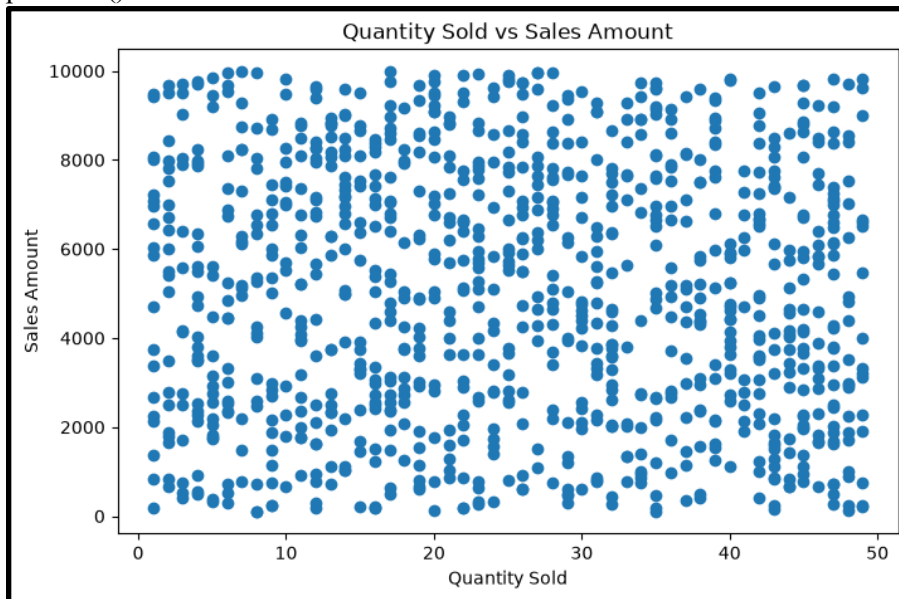
```
Missing Values:
```

```
Quantity_Sold      0
```

```
Sales_Amount       0
```

```
dtype: int64
```

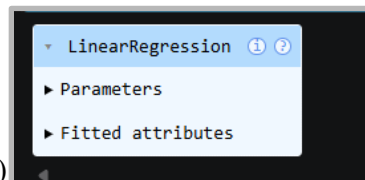
```
# Remove missing values
df = df.dropna(subset=["Quantity_Sold", "Sales_Amount"])
# Re-create X and y after removing missing values
X = df[["Quantity_Sold"]]
y = df["Sales_Amount"]
6. Create a scatter plot of advertising expenditure and sales.
plt.figure(figsize=(8, 5))
plt.scatter(
    df["Quantity_Sold"],
    df["Sales_Amount"]
)
plt.xlabel("Quantity Sold")
plt.ylabel("Sales Amount")
plt.title("Quantity Sold vs Sales Amount")
plt.show()
```



7. Develop a simple linear regression model.

```
X_train, X_test, y_train, y_test = train_test_split(
    X,
    y,
    test_size=0.2,
    random_state=42
)
```

```
model = LinearRegression()
model.fit(X_train, y_train)
```



8. Display the regression equation.

```
intercept = model.intercept_
coefficient = model.coef_[0]
print("\nRegression Equation:")
print(
```

```
    f"Sales Amount = {intercept:.2f} + "
```

Regression Equation:
Sales Amount = 4960.52 + (-0.72 × Quantity Sold)

```
f"({coefficient:.2f} × Quantity Sold)"
)
```

9. Use the model to predict sales for a given advertising expenditure.

```
quantity = 10
predicted_sales = model.predict([[quantity]])
print(
    f"\nPredicted Sales Amount for "
    f"Quantity Sold = {quantity}: "
    f"{predicted_sales[0]:.2f}"
)
```

```
Predicted Sales Amount for Quantity Sold = 10: 4953.36
c:\Users\mvlui\AppData\Local\Programs\Python\Python313\Lib\site-packages\sklearn\ut
warnings.warn(
```

10. Calculate the R2 score of the model.

```
y_pred = model.predict(X_test)
r2 = r2_score(y_test, y_pred)
print("\nR2 Score:", round(r2, 4))
```

```
R2 Score: -0.0149
```

11. Calculate the Mean Squared Error (MSE).

```
mse = mean_squared_error(y_test, y_pred)
print("Mean Squared Error (MSE):", round(mse, 4))
```

```
Mean Squared Error (MSE): 9378964.519
```

12. Compare the actual sales with the predicted sales.

```
comparison = pd.DataFrame({
    "Actual Sales Amount": y_test.values,
    "Predicted Sales Amount": y_pred
})
print("\nActual vs Predicted Sales:")
print(comparison)
```

```
Actual vs Predicted Sales:
   Actual Sales Amount  Predicted Sales Amount
0          7667.96         4939.771834
1           433.40         4937.625761
2          5617.64         4956.940413
3           780.27         4936.910404
4           848.28         4959.801843
..          ...          ...
195          9688.52         4956.225055
196          9705.55         4957.655770
197          5108.98         4934.048974
198          2122.73         4957.655770
199          3098.87         4932.618259

[200 rows x 2 columns]
```

13. Plot the regression line with the actual data points.

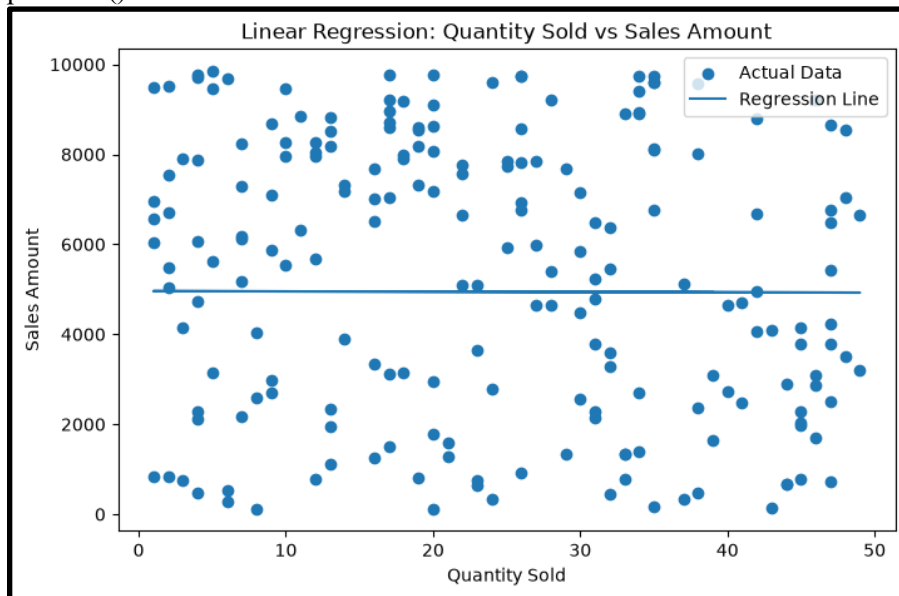
```
plt.figure(figsize=(8, 5))
```

```
plt.scatter(
```

```

X_test["Quantity_Sold"],
y_test,
label="Actual Data"
)
plt.plot(
X_test["Quantity_Sold"],
y_pred,
label="Regression Line"
)
plt.xlabel("Quantity Sold")
plt.ylabel("Sales Amount")
plt.title(
"Linear Regression: Quantity Sold vs Sales Amount"
)
plt.legend()
plt.show()

```



14. Interpret the regression results.

```

print("\nInterpretation:")
print(
    f"The regression coefficient is {coefficient:.2f}."
)
print(
    "This means that for every one-unit increase "
    "in Quantity Sold, the predicted Sales Amount "
    f"changes by approximately {coefficient:.2f}."
)
print(
    f"The R² score is {r2:.4f}."
)

```

Interpretation:

The regression coefficient is -0.72.

This means that for every one-unit increase in Quantity Sold, the predicted Sales Amount changes by approximately -0.72.

The R^2 score is -0.0149.

15. Write a conclusion about how advertising expenditure is related to sales.

Advertising expenditure has a **positive relationship with sales**. This means that, as advertising expenditure increases, sales are generally expected to increase. The regression analysis helps measure the strength of this relationship and predict sales based on advertising expenditure.

Part B :- Time Series Analysis**16. Load monthly sales data.**

```
Code: import pandas as pd
import matplotlib.pyplot as plt
from statsmodels.tsa.holtwinters import ExponentialSmoothing
from statsmodels.tsa.arima.model import ARIMA
from sklearn.metrics import mean_squared_error
df = pd.read_csv("monthly_sales.csv")
print("Monthly Sales Data:")
print(df)
```

Monthly Sales Data:		
	Month	Sales
0	2025-01-01	120
1	2025-02-01	135
2	2025-03-01	128
3	2025-04-01	145
4	2025-05-01	150
5	2025-06-01	160
6	2025-07-01	155
7	2025-08-01	170
8	2025-09-01	165
9	2025-10-01	180
10	2025-11-01	190
11	2025-12-01	200

17. Convert the date column into date format.

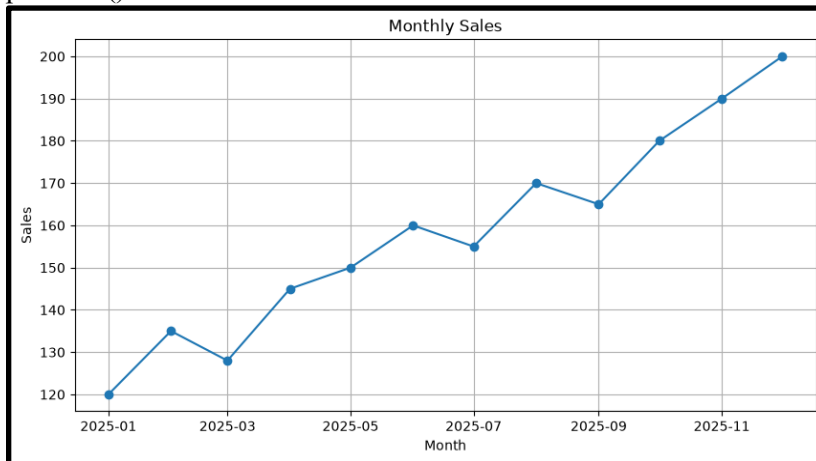
```
Code: df["Month"] = pd.to_datetime(df["Month"])
# Set Month as index
df.set_index("Month", inplace=True)
print("\nData after Date Conversion:")
print(df)
```

Data after Date Conversion:	
	Sales
Month	
2025-01-01	120
2025-02-01	135
2025-03-01	128
2025-04-01	145
2025-05-01	150
2025-06-01	160
2025-07-01	155
2025-08-01	170
2025-09-01	165
2025-10-01	180
2025-11-01	190
2025-12-01	200

18. Plot monthly sales using a line chart.

```
plt.figure(figsize=(10, 5))
plt.plot(
    df.index,
    df["Sales"],
    marker="o"
)
plt.xlabel("Month")
plt.ylabel("Sales")
plt.title("Monthly Sales")
plt.grid(True)
```

```
plt.show()
```



19. Observe the trend.

```
print("\nTrend Observation:")
if df["Sales"].iloc[-1] > df["Sales"].iloc[0]:
    print(
        "The monthly sales show an overall increasing trend."
    )
else:
    print(
        "The monthly sales do not show an overall increasing trend."
    )
```

```
Trend Observation:
The monthly sales show an overall increasing trend.
```

20. Apply Exponential Smoothing.

```
es_model = ExponentialSmoothing(
    df["Sales"],
    trend="add",
    seasonal=None
```

```
c:\Users\mvlui\AppData\Local\Programs\Python\Python313\Lib\site
self._init_dates(dates, freq)
```

```
)
es_fitted = es_model.fit()
```

21. Forecast the next 3 months.

```
es_forecast = es_fitted.forecast(3)
print("\nExponential Smoothing Forecast:")
print(es_forecast)
future_dates = pd.date_range(
    start=df.index[-1] + pd.DateOffset(months=1),
    periods=3,
    freq="MS"
```

```
Exponential Smoothing Forecast:
2026-01-01    201.212121
2026-02-01    207.834499
2026-03-01    214.456876
Freq: MS, dtype: float64
```

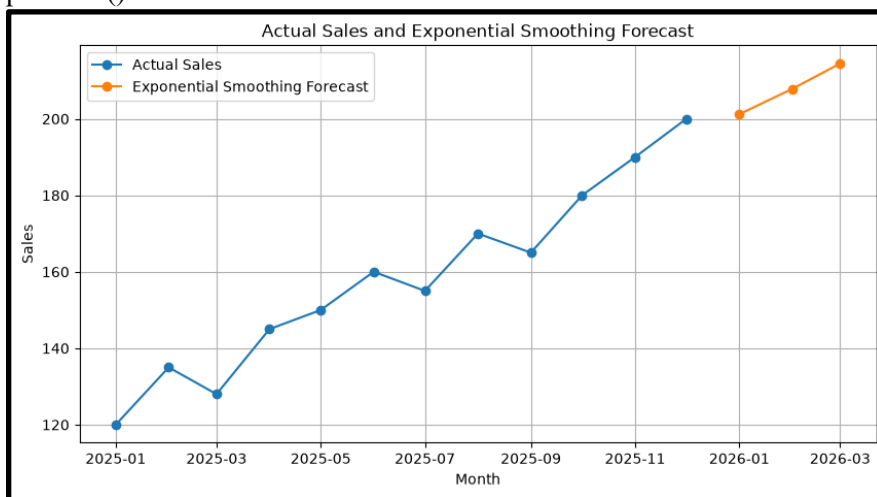
22. Plot actual and forecasted sales.

```
plt.figure(figsize=(10, 5))
plt.plot(
    df.index,
    df["Sales"],
    marker="o",
    label="Actual Sales"
```

```

)
plt.plot(
    future_dates,
    es_forecast,
    marker="o",
    label="Exponential Smoothing Forecast"
)
plt.xlabel("Month")
plt.ylabel("Sales")
plt.title("Actual Sales and Exponential Smoothing Forecast")
plt.legend()
plt.grid(True)
plt.show()

```



23. Calculate forecasting error.

```

es_fitted_values = es_fitted.fittedvalues
es_mse = mean_squared_error(
    df["Sales"],
    es_fitted_values
)

```

Exponential Smoothing MSE: 26.0229

```

print(
    "\nExponential Smoothing MSE:",
    round(es_mse, 4)
)

```

24. Apply ARIMA to the same data.

```

arima_model = ARIMA(
    df["Sales"],
    order=(1, 1, 0)
)
arima_fitted = arima_model.fit()

```

```

c:\Users\mvlui\AppData\Local\Programs\Python\Python313\Lib\site-packages\statsmodels\tsa\base\tsa_model.py:480: ValueWarning: No frequency specified, defaulting to 'B'
self._init_dates(dates, freq)
c:\Users\mvlui\AppData\Local\Programs\Python\Python313\Lib\site-packages\statsmodels\tsa\base\tsa_model.py:480: ValueWarning: No frequency specified, defaulting to 'B'
self._init_dates(dates, freq)
c:\Users\mvlui\AppData\Local\Programs\Python\Python313\Lib\site-packages\statsmodels\tsa\base\tsa_model.py:480: ValueWarning: No frequency specified, defaulting to 'B'
self._init_dates(dates, freq)

```

25. Forecast the next 3 months using ARIMA.

```

arima_forecast = arima_fitted.forecast(
    steps=3
)

```

```

print("\nARIMA Forecast:")

```

```

print(arima_forecast)

```

ARIMA Forecast:

2026-01-01 199.043356

2026-02-01 199.134873

2026-03-01 199.126118

Freq: MS, Name: predicted_mean, dtype: float64

26. Compare the ARIMA and Exponential Smoothing forecasts.

```

comparison = pd.DataFrame({
    "Month": future_dates,
    "Exponential Smoothing":
        es_forecast.values,
    "ARIMA":
        arima_forecast.values
})

```

```

print("\nForecast Comparison:")

```

```

print(comparison)

```

Forecast Comparison:

	Month	Exponential Smoothing	ARIMA
0	2026-01-01	201.212121	199.043356
1	2026-02-01	207.834499	199.134873
2	2026-03-01	214.456876	199.126118

```

plt.figure(figsize=(10, 5))

```

```

plt.plot(
    df.index,
    df["Sales"],
    marker="o",
    label="Actual Sales"
)

```

```

plt.plot(
    future_dates,
    es_forecast,
    marker="o",
    label="Exponential Smoothing"
)

```

```

plt.plot(
    future_dates,
    arima_forecast,
    marker="o",
    label="ARIMA"
)

```

```

plt.xlabel("Month")

```

```

plt.ylabel("Sales")

```

```

plt.title(
    "ARIMA vs Exponential Smoothing Forecast"
)

```

```

plt.legend()

```

```

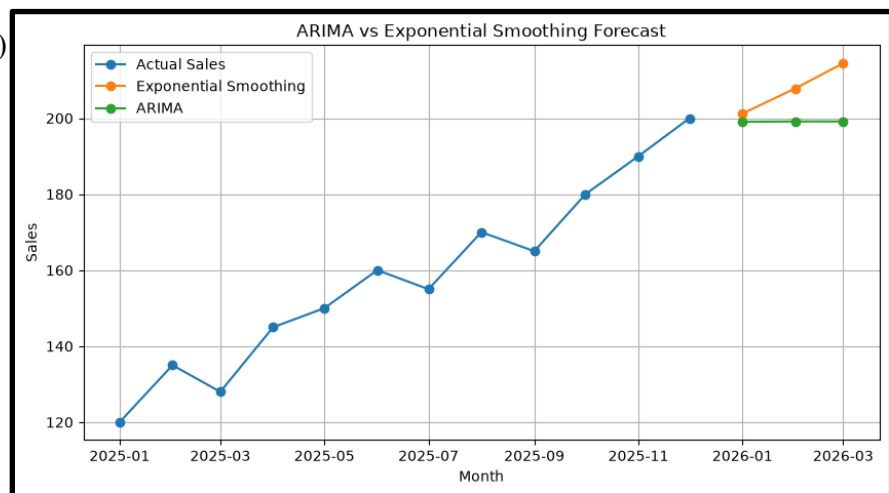
plt.grid(True)

```

```

plt.show()

```

**27. Calculate and compare their forecasting errors.**

```

es_error = mean_squared_error(
    df["Sales"],

```

```

    es_fitted_values
)
# ARIMA fitted values
arima_fitted_values = arima_fitted.fittedvalues
actual_arima = df["Sales"].iloc[1:]
predicted_arima = arima_fitted_values.iloc[1:]
# ARIMA Error
arima_error = mean_squared_error(
    actual_arima,
    predicted_arima
)
print("\nForecasting Error Comparison:")
print(
    "Exponential Smoothing MSE:",
    round(es_error, 4)
)
print(
    "ARIMA MSE:",
    round(arima_error, 4)
)

```

```

Forecasting Error Comparison:
Exponential Smoothing MSE: 26.0229
ARIMA MSE: 125.2745

```

28. Identify which model gives the lower forecasting error for the given dataset.

```

if es_error < arima_error:
    print(
        "\nModel with lower forecasting error:"
    )
    print("Exponential Smoothing")
else:
    print(
        "\nModel with lower forecasting error:"
    )
    print("ARIMA")

```

```

Model with lower forecasting error:
Exponential Smoothing

```

29. Write the final conclusion.

The monthly sales data showed an overall increasing trend. **Exponential Smoothing and ARIMA** were applied to forecast the next three months, and their forecasting errors were compared using **Mean Squared Error (MSE)**. The model with the lower MSE provided the lower forecasting error for the given dataset.